

TOPIC 6

Timing Constraints and Analysis

Introduction to timing constraints in RTL design · Timing analysis and optimization techniques · Setup and hold time violations and resolution

- 6a · Constraints — what the tool cannot know until you tell it
- 6b · Analysis — how static timing analysis actually computes slack
- 6c · Violations — setup and hold, why they are opposite, and how to fix each
- Tools · Vivado · OpenSTA · a 400-line STA engine you build · Labs T1-T7



What This Topic Covers, and Why Every Chip Depends On It

Topic 4 taught you to write RTL. Topic 5 taught you to prove it is **functionally** correct. Topic 6 asks the last question before silicon: **is it fast enough, and will it work at all?** A design that simulates perfectly can still fail on the bench - because simulation does not know how long a wire takes.

Syllabus bullet — Topic 6	Covered by	Slides
Introduction to timing constraints in RTL design	6a	5-26
Timing analysis and optimization techniques	6b	27-48
Setup and hold time violations and resolution	6c	49-68
Practical: Timing Analysis and Closure Labs (10 h)	Labs T1-T7	69-78
Practical: Design Synthesis and Optimisation Labs (15 h)	Labs T5-T7	74-78

Learning outcomes — by the end of this topic you can

- Write a complete SDC/XDC constraint file and defend every line in it.
- Compute arrival, required and slack by hand, then verify a tool's answer against your own.
- Read any timing report - Vivado, OpenSTA, PrimeTime - and find the problem in under a minute.
- Diagnose a setup violation and pick the cheapest fix that works.
- Recognise a hold violation, explain why slowing the clock cannot help, and fix it properly.

One sentence to open the session with

A functional bug makes the chip do the wrong thing. A timing bug makes it do the right thing at the wrong moment - and no amount of simulation will show you that.



WHY THIS MATTERS

The Bug That Simulation Cannot Find

Your testbench applies a value, waits for a clock edge, and checks the answer. It assumes the logic finished in time. **Real gates take real time.** If the answer is 0.2 ns late, the flop captures the OLD value - and your simulation, which had no delays in it, never noticed.

What RTL simulation checks

Does the design compute the right value, given unlimited time?

Every functional bug you found in Topic 5 lives here.

Zero-delay: every assignment lands instantly.

What static timing analysis checks

Does that value ARRIVE before the next clock edge?

And does it stay put long enough to be captured?

Real gate delays, real wire delays, real clock skew.

What neither one checks

Whether your CONSTRAINTS describe the real system.

A wrong period or a missing input delay makes both tools lie in the same direction: everything looks fine.

The order of operations, and it is not negotiable

1. Simulate until the function is right. 2. Constrain until the report is honest. 3. Analyse until the slack is positive. **4. Only then build.**

Skip step 2 and step 3 becomes theatre: the tool reports MET on paths it never examined.

PART 6a

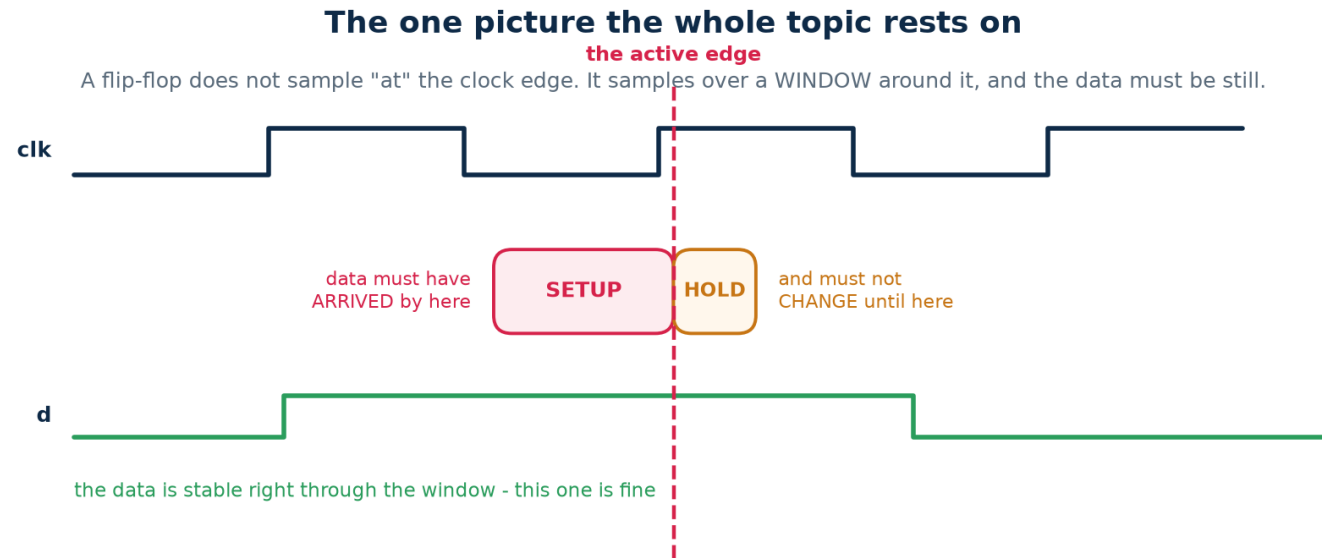
Timing Constraints in RTL Design

Synthesis has no idea how fast you want to go until you tell it. This part is about telling it - completely, and truthfully.

- The clock: `create_clock`, `period`, `waveform`, `uncertainty`
- The boundary: `set_input_delay` and `set_output_delay`
- The exceptions: `set_false_path` and `set_multicycle_path`
- The order to write a constraint file, and the holes to check for



A Flip-Flop Samples Over a Window, Not At an Instant



Break either half and you get a different failure

SETUP violated

the data arrived too LATE.
Fix: a faster path, or a slower clock.

HOLD violated

the data changed too SOON.
Fix: a SLOWER path. The clock cannot help.

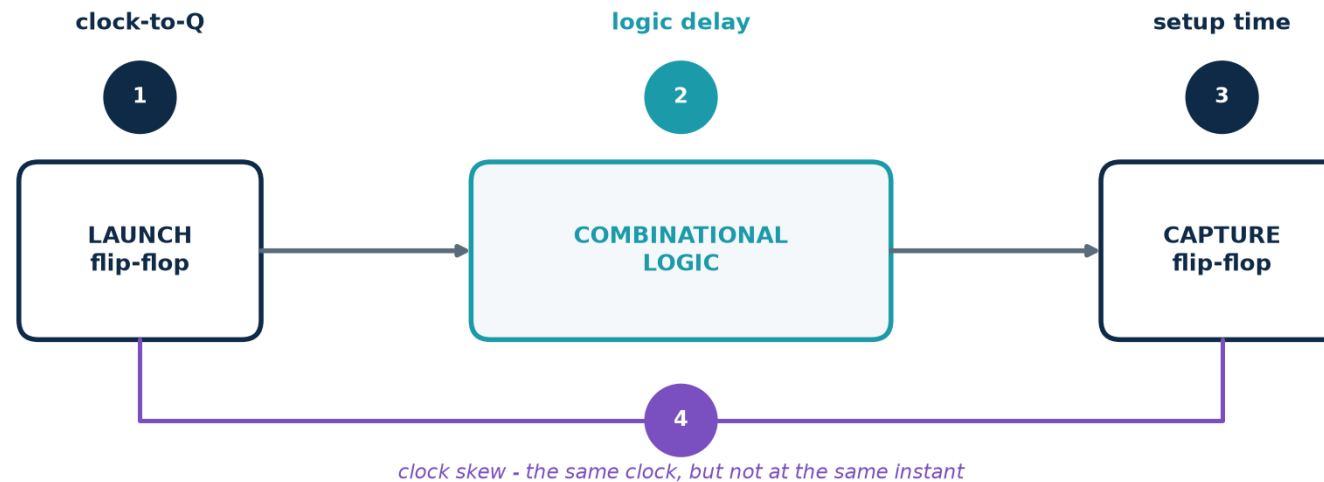
Every timing concept in this topic comes from this one diagram

SETUP time: the data must be stable for this long BEFORE the edge. HOLD time: it must stay stable for this long AFTER it. Break the first and the flop captures a stale value; break the second and it captures the NEXT value, a whole cycle early.



Every Path Has Exactly Four Parts

Every timing path has exactly four parts



and the equation that uses all four

$$\text{setup slack} = \left(\text{period} + \text{skew} - \text{setup} - \text{uncertainty} \right) - \left(\text{clock-to-Q} + \text{logic delay} \right)$$

| _____ required time _____ | | _____ arrival time _____ |

Learn these four names and the rest of the topic is arithmetic

clock-to-Q (how long the launching flop takes to produce Q) · logic delay (the combinational path) · setup time (how early the capturing flop needs it) · clock skew (the two flos do not see the edge at the same instant)



Slack Is a Subtraction, and That Is All It Is

Slack is a subtraction, and that is all it is

ARRIVAL

when the data actually turns up



REQUIRED

when it had to be there by



SLACK

Positive slack means it fits. Negative slack means it does not, and the number is by how much.

WNS is the WORST slack in the design. TNS adds up every negative slack, so it says how BIG the problem is, not just how bad.

the only equation you must memorise

$\text{slack} = \text{required time} - \text{arrival time}$

$\text{arrival} = \text{clock-to-Q} + \text{logic delay}$ (when the data turns up)

$\text{required} = \text{period} + \text{skew} - \text{setup} - \text{uncertainty}$ (when it had to be there)

Positive slack: it fits, with room to spare. Negative slack: it does not fit, and the number is by how much.



A Tool With No Constraints Is Not Careful - It Is Blind

A tool with no constraints is not being careful - it is being blind

Synthesis and place-and-route optimise for the goal you give them. No goal, no optimisation.

WITHOUT constraints

- the tool assumes an infinitely fast clock
- every path is "good enough" - none is worked on
- the report says MET because nothing was checked
- the chip fails on the bench, not on the screen

WITH constraints

- the tool knows the target period
- it spends effort on the paths that fail
- the report is a real prediction of silicon
- "MET" means something you can trust

The constraint file is a specification, not a settings file

It says how fast the clock runs, when data arrives from outside, when it must leave, and which paths are not real. Everything the timing report says is downstream of it.

The single most dangerous report in engineering

A timing report with no violations on an unconstrained design. It is not telling you the design is fast. It is telling you it did not look.



Every Constraint File Answers the Same Four Things

Four questions - and the constraint that answers each

1

How fast does the clock run?

```
create_clock -period 10 \  
-name clk [get_ports clk]
```

2

When does data arrive from outside?

```
set_input_delay 3.0 \  
-clock clk [get_ports din*]
```

3

When must data leave the chip?

```
set_output_delay 2.5 \  
-clock clk [get_ports dout*]
```

4

Which paths are not real?

```
set_false_path / \  
set_multicycle_path
```

Every timing report you will ever read is an answer to these four questions.

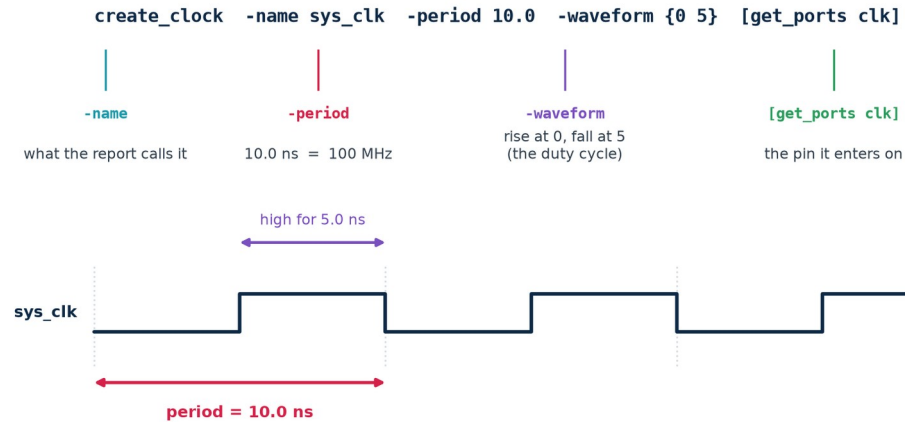
SDC is the vendor-neutral name (Synopsys Design Constraints). Xilinx calls its version XDC - the timing commands are the same.

SDC (Synopsys Design Constraints) is the vendor-neutral format. Xilinx XDC is SDC plus physical constraints. Intel Quartus reads SDC directly. Learn it once.



The One Constraint You Cannot Leave Out

create_clock - the one constraint you cannot leave out



Get the period wrong and every number in the report is wrong by the same amount.

clocks, primary and generated

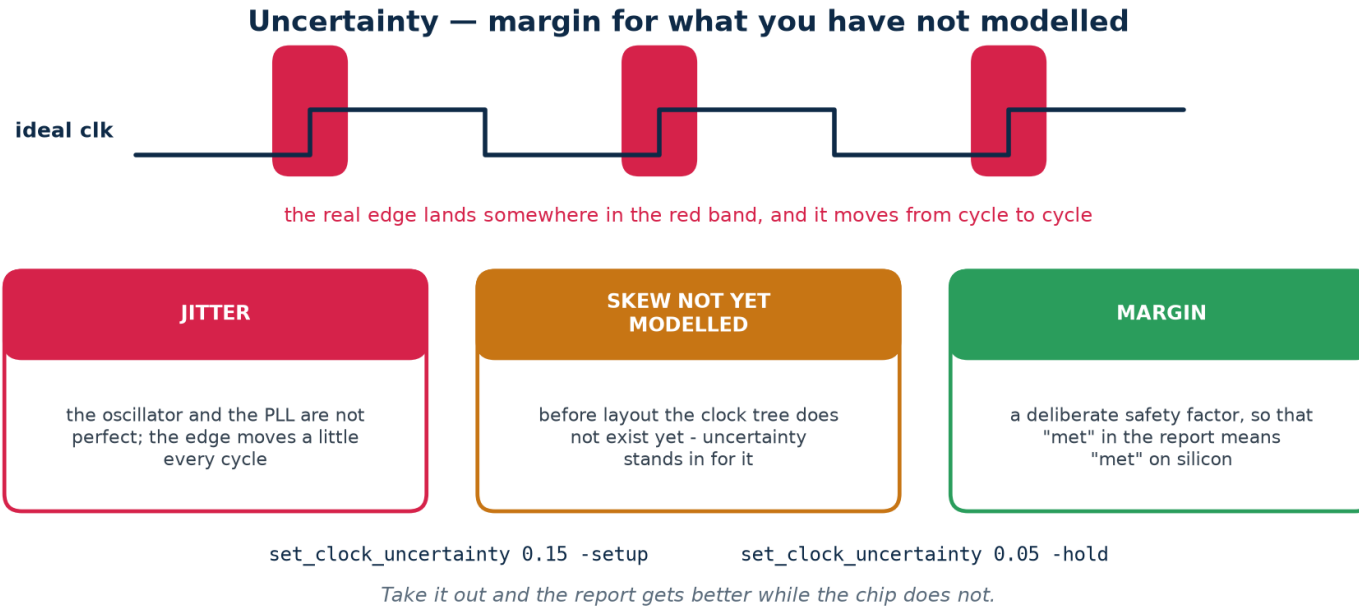
```
# a 100 MHz primary clock arriving on the port called clk
create_clock -name sys_clk -period 10.000 -waveform {0.000 5.000} [get_ports clk]

# a clock made by a divide-by-2 inside the design - derive it, never re-declare it
create_generated_clock -name clk_div2 -source [get_ports clk] \
    -divide_by 2 [get_pins div_reg/Q]
```

If a register is clocked by something you never declared, its paths are unconstrained and silently unchecked.



Margin For What You Have Not Modelled Yet



before layout, uncertainty stands in for the clock tree

```
set_clock_uncertainty 0.150 -setup [get_clocks sys_clk] # jitter + unbuilt skew
set_clock_uncertainty 0.050 -hold [get_clocks sys_clk] # jitter only
```

A number you will be tempted to reduce, and should not

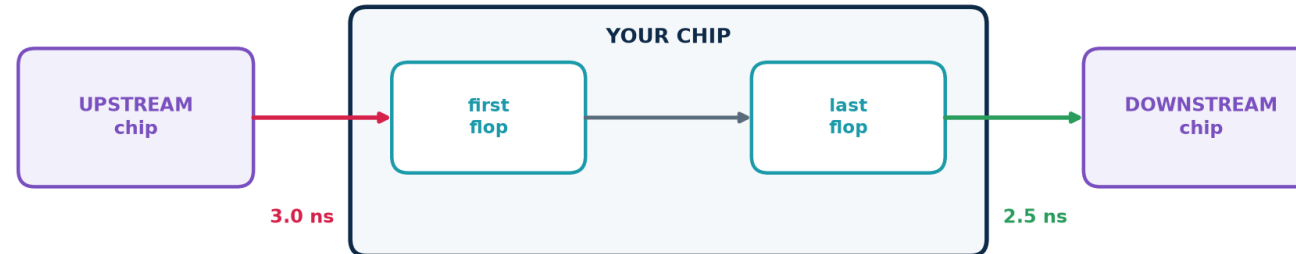
Uncertainty is the difference between "the report says MET" and "the silicon works". Shrinking it makes the report prettier and the chip no faster. Typical FPGA practice: 0.1-0.2 ns pre-layout, then let the tool use the real clock tree afterwards.



Your First Flop Is Not the Start of the Path

Input and output delay - the part of the path outside your chip

Your first flop is not the start of the path. The path started in another chip.



set_input_delay 3.0

"the data leaves the other chip 3.0 ns after the same clock edge, so I only have period - 3.0 left for my own logic."

set_output_delay 2.5

"the next chip needs the data 2.5 ns before the edge, so I must be finished 2.5 ns early."

The most common constraint bug in the world

Forget these two lines and every I/O path becomes UNCONSTRAINED. The report shows no violation because it never looked. Always check the unconstrained-endpoint count.

The most common constraint bug in the world

Forget these two lines and every path touching a pin becomes UNCONSTRAINED. The report shows no violation because it never looked at them. Always check the unconstrained-endpoint count before you believe a clean report.



Writing Them - and Where the Numbers Come From

constraints/io.sdc

```
# INPUT: how late the data arrives, measured from the SAME clock edge
set_input_delay -clock sys_clk -max 3.0 [get_ports {din[*] valid}]
set_input_delay -clock sys_clk -min 0.5 [get_ports {din[*] valid}]

# OUTPUT: how much of the period the NEXT chip needs, before its own edge
set_output_delay -clock sys_clk -max 2.5 [get_ports {dout[*] ready}]
set_output_delay -clock sys_clk -min 0.2 [get_ports {dout[*] ready}]

# a quick sanity sweep: constrain everything that is still bare
set_input_delay -clock sys_clk 3.0 [remove_from_collection \
    [all_inputs] [get_ports clk]]
```

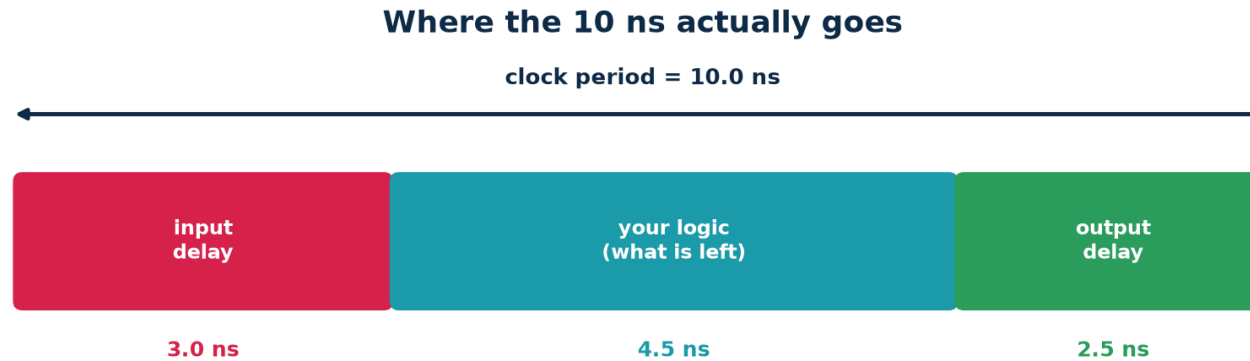
-max is used for	-min is used for	Where the number comes from
the SETUP check	the HOLD check	the upstream chip's datasheet: clock-to-out
the slow corner	the fast corner	plus the board trace delay
the LATEST the data can arrive	the EARLIEST it can change	when unknown, budget 30% of the period and revisit

A rule of thumb that has saved many projects

Register every input and every output. It costs one cycle of latency and removes the entire I/O timing problem: the path from pin to flop becomes short and the path from flop to pin becomes short. Do it unless you have a measured reason not to.



Where the 10 Nanoseconds Actually Goes



Read that middle bar again

You asked for 100 MHz, but your own logic only gets 4.5 ns - the I/O ate more than half the cycle.

If the design will not close, look here before you touch the RTL. Registering the pins moves the I/O delay into its own cycle and hands the whole 10 ns back to the logic.

Read the middle bar before you touch the RTL

You asked for 100 MHz and your own logic got 4.5 ns of it. If the design will not close, the I/O budget is the first place to look - and registering the pins hands the whole period back.



Telling the Tool a Path Is Not What It Looks Like

Exceptions - telling the tool a path is not what it looks like

By default the tool checks EVERY path in one cycle. Sometimes that is simply not the design.

set_false_path

"do not check this path at all"

a reset that is only ever released,
a test-mode pin, a path that cannot
physically be exercised

set_multicycle_path

"this one gets N cycles, not one"

a wide multiply whose result is only
read every fourth cycle, a slow
configuration register

An exception is a PROMISE, and the tool believes you

set_false_path on a path that IS real removes the only check that would have caught the bug.
The report goes green and the silicon goes wrong. This is how real chips die.

Write the exception only when you can say, in one sentence, why the path cannot matter.

```
set_multicycle_path 4 -setup -from [get_cells a_q*] -to [get_cells acc*]
```

```
set_multicycle_path 3 -hold -from [get_cells a_q*] -to [get_cells acc*]
```

The hold number is almost always N-1. Forget the hold line and you have just created a hold violation.

An exception is a promise, and the tool believes you without checking. Every one needs a comment saying why it is true.



When "Do Not Check This" Is the Honest Answer

the three legitimate uses

```
# a reset that is asserted asynchronously and released synchronously:  
# the assertion edge is never timed, the release IS - so cut only one direction  
set_false_path -from [get_ports rst_n]  
  
# a mode pin written once at power-up and never again  
set_false_path -from [get_cells cfg_mode_reg]  
  
# two clocks that are genuinely unrelated (no data crosses without a synchroniser)  
set_clock_groups -asynchronous -group [get_clocks sys_clk] \  
                 -group [get_clocks usb_clk]
```

Legitimate

The path physically cannot be exercised.
A synchroniser already handles the crossing.
The endpoint is not sampled by that clock at all.

Not legitimate

"It always fails and I do not know why."
"The tool is being pessimistic."
"We ran out of time before tape-out."

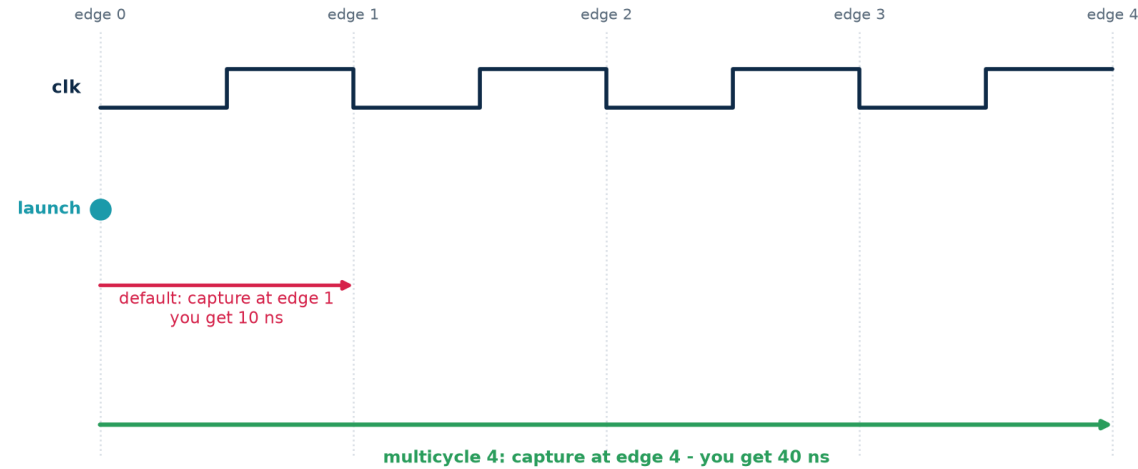
How this kills a chip

set_false_path on a path that IS real removes the only check that would have caught the bug. The report goes green, the design ships, and the failure appears in the field at one temperature and one voltage. Write the justification in the file, next to the line.



When One Cycle Was Never the Requirement

What "multicycle 4" changes: which edge does the capturing



Nothing in the hardware changed. You told the tool the truth about when the result is read.

The design must actually hold the input steady for those four cycles - usually an enable on the capture flop.

setup N, hold N-1 - and the hold line is not optional

```
set_multicycle_path 4 -setup -from [get_cells a_q*] -to [get_cells acc*]  
set_multicycle_path 3 -hold -from [get_cells a_q*] -to [get_cells acc*]
```

Why the hold number is N-1

The setup line moves the CAPTURE edge four cycles later. Left alone, the hold check would then compare against that same distant edge and demand an absurd amount of delay. The -hold N-1 line moves the hold check back to where it belongs: one edge after the launch.



The Part Everyone Forgets: the Hardware Must Agree

rtl/slow_path.v — the enable that justifies the exception

```
// the SDC exception is a claim about this design. Make the claim true:
reg [1:0] phase;
always @(posedge clk) phase <= phase + 1'b1;
wire tick = (phase == 2'b11);          // one cycle in four

always @(posedge clk) begin
    if (tick) acc <= a_q + b_q;        // the ENABLE is what makes it legal
end

// a_q and b_q must also be held steady across all four cycles -
// if they change every cycle, the multicycle claim is simply false.
```

Without the exception	With it	Measured in the lab
worst slack -1.193 ns	worst slack +0.392 ns	one SDC line
the tool times a 32-bit ripple add in one cycle	it times it across four	no RTL changed at all
you pipeline something that never needed it	you ship what you already had	hours of work avoided

The discipline

Never write a multicycle path you cannot point at the enable for. If the capture register has no enable, the exception is a lie and the silicon will find it.



The Order To Write It - and the Order To Debug It

The order to write a constraint file - and the order to debug it

1

Define every clock

create_clock for each primary clock; create_generated_clock for anything a divider makes.

2

Add uncertainty

set_clock_uncertainty for jitter and, before layout, for the skew the clock tree does not have yet.

3

Constrain the I/O

set_input_delay and set_output_delay on every port, against the clock that samples it.

4

Only now, exceptions

set_false_path and set_multicycle_path - each one justified in a comment on the line above it.

5

Check for holes

Report unconstrained endpoints. A clean report on a half-constrained design is worse than a failing one.

When a timing report surprises you, suspect the constraints before you suspect the RTL.

The check that catches everything else

Report unconstrained endpoints on every run. A design with 400 of them and WNS +2.0 ns is in far worse shape than one with zero and WNS -0.1 ns - the first one has not been analysed at all.



Everything Above, In One Real Constraint File

```
# ===== constraints/add32.sdc
# clock -----
create_clock -name clk -period 5.000 [get_ports clk]
set_clock_uncertainty 0.100 -setup [get_clocks clk]
set_clock_uncertainty 0.030 -hold [get_clocks clk]

# boundary -----
set_input_delay -clock clk -max 1.20 [get_ports {a[*] b[*]}]
set_input_delay -clock clk -min 0.30 [get_ports {a[*] b[*]}]
set_output_delay -clock clk -max 1.00 [get_ports {sum[*] cout}]
set_output_delay -clock clk -min 0.20 [get_ports {sum[*] cout}]

# exceptions -----
# cfg_mode is written by software at boot and read only when the core is halted.
set_false_path -from [get_cells cfg_mode_reg]

# environment -----
set_max_fanout 16 [current_design]
```



Ten Questions Before We Move On

#	Question	The answer in one line
1	What does create_clock actually do?	gives the tool a target; without it there is none
2	What is the unit of slack?	time - nanoseconds, signed
3	Positive slack means?	the path fits, with that much to spare
4	What does set_input_delay describe?	the part of the path outside your chip
5	What happens if you omit it?	those paths become unconstrained - unchecked
6	Why is uncertainty there?	jitter, and skew that does not exist yet
7	When is a false path legitimate?	when the path cannot be exercised, and you can say why
8	Multicycle 4 setup needs what hold?	3 - almost always N-1
9	What must the RTL have for it?	an enable that really does hold the capture
10	What do you check last, every time?	the unconstrained-endpoint count

If any of these is still fuzzy, the analysis in part 6b will not land. Ask now.

PART 6b

Timing Analysis and Optimization Techniques

How a tool turns a netlist into a slack number - and what to do when that number is negative.

- Static timing analysis: the graph, the two sweeps, the subtraction
- Reading a timing report line by line
- Path groups, WNS, TNS, Fmax and PVT corners
- Optimisation: constraints, mapping, restructuring, pipelining, retiming



Static Timing Analysis Replaced Timing Simulation

Why STA replaced timing simulation

TIMING SIMULATION

runs vectors through a delay model

- needs stimulus you have to write
- only checks the paths your vectors happen to exercise
- slow - hours for a medium design
- silent about the path you forgot

STATIC TIMING ANALYSIS

walks the graph, no vectors at all

- no stimulus needed
- checks EVERY path, exhaustively
- seconds to minutes
- tells you the worst one by name

"Static" means it does not care what the values are

STA never asks whether a path can be activated - only how long it is. That is its strength (nothing is missed) and its one weakness (a path that can never switch is still reported).

The paths that can never switch are called FALSE PATHS. Removing them is your job, not the tool's - which is exactly why `set_false_path` exists.

The word "static" is the whole idea

STA never asks what the data values are. It only asks how long the path is. That is why it needs no stimulus, why it cannot miss a path - and why it will happily report a path that can never actually switch.

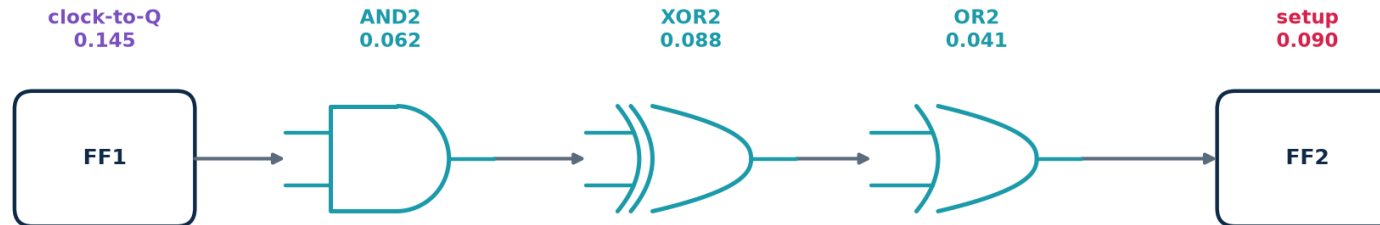


6b · STEP 1

The Netlist Becomes a Graph of Delays

Step 1 - the netlist becomes a graph of delays

Every pin is a node. Every cell and every wire is an edge with a number on it.



$$\text{arrival at FF2/D} = 0.145 + 0.062 + 0.088 + 0.041 = 0.336 \text{ ns}$$

Three rules that make the whole thing work

1. Arrival time flows FORWARD. At a node with several inputs, keep the LATEST (for setup).
2. Required time flows BACKWARD from the capture flop, starting at period - setup.
3. slack = required - arrival, computed at every endpoint. Negative means it does not fit.

the delay model used throughout this topic

$$\text{cell delay} = \text{intrinsic} + \text{load_factor} \times (\text{sum of input caps on the output net})$$

Real Liberty libraries use a 2-D table indexed by input slew and output load. The arithmetic below is the same; only the lookup is bigger.



A Liberty Cell, In Full

lib/cda_edu.lib — you write this file in lab T1

```
cell (XOR2) {  
    area          : 4.0 ;  
    cda_intrinsic  : 0.088 ;      /* ns, with no load at all      */  
    cda_load_factor : 0.018 ;      /* ns per unit of capacitance on Y */  
    cda_input_cap   : 1.5 ;      /* what this cell presents to its driver */  
    pin (A) { direction : input ; }  
    pin (B) { direction : input ; }  
    pin (Y) { direction : output ; function : "(A^B)" ; }  
}  
  
cell (DFF) {  
    cda_clk_to_q    : 0.145 ;      /* CK rising -> Q valid      */  
    cda_setup       : 0.090 ;      /* D stable before CK        */  
    cda_hold        : 0.035 ;      /* D stable after CK         */  
    cda_input_cap   : 1.6 ;  
}
```

Why you write the library yourself

Every delay number in this topic is one you chose. When the report says 0.164 ns you can point at the two lines of the library it came from. There is no black box anywhere in the chain.



6b · WORKED EXAMPLE

One Path, By Hand, To Three Decimal Places

path: p_reg/Q -> u12/A -> u12/Y -> q_reg/D

1. p_reg/Q clk-to-Q 0.145, driving a net loaded by u12/A (cap 1.5)
 $0.145 + 0.013 \times 1.5 = 0.1645$ <- DFF load_factor 0.013

2. u12 (X0R2) intrinsic 0.088, driving a net loaded by q_reg/D (cap 1.6)
 $0.088 + 0.018 \times 1.6 = 0.1168$

arrival at q_reg/D = $0.1645 + 0.1168 = 0.2813$ ns

3. required = period 1.000 - setup 0.090 - uncertainty 0.000 = 0.910 ns

slack = $0.910 - 0.281 = +0.629$ ns MET

This is not an illustration - it is the lab's actual output

Run `make tiny` and the engine prints `+0.629`. You will have computed it on paper first. If the two ever disagree, one of them has a bug, and finding out which is the most useful hour in the topic.



6b · STEP 2

Two Sweeps, Then One Subtraction

Step 2 - two sweeps, then one subtraction

FORWARD SWEEP - arrival time



BACKWARD SWEEP - required time



and now the subtraction, at the endpoint

$$\text{slack} = \text{required } 0.960 - \text{arrival } 0.336 = +0.624 \text{ ns MET}$$

Do that at every endpoint and you have the whole report. The worst answer is the WNS.
A design with a million flops is a million of these subtractions - which is why STA is fast.

Forward for arrival, backward for required, subtract at every endpoint. A million-flop design is a million of these subtractions - which is why STA finishes in minutes.



Fifteen Lines That Do the Whole Job

sta/sta.py — the core of the engine you complete in lab T3

```
def propagate(self):
    for n in self._order():          # topological order
        for arc in n.fanin:
            src, dly = arc.src, arc.delay
            n.amax = max(n.amax, src.amax + dly)    # LATEST -> setup
            n.amin = min(n.amin, src.amin + dly)    # EARLIEST-> hold

def setup_slack(self, ep):
    required = self.period + self.skew(ep) - ep.cell.setup - self.uncert_setup
    return required - ep.amax

def hold_slack(self, ep):
    required = self.skew(ep) + ep.cell.hold + self.uncert_hold
    return ep.amin - required

# WNS = min(setup_slack(ep) for ep in endpoints)
# TNS = sum(s for s in slacks if s < 0)
```



Why the Engine Carries Two Numbers Per Node

	Setup check (max delay)	Hold check (min delay)
propagates	the LATEST arrival	the EARLIEST arrival
asks	did it get there in time?	did it stay put long enough?
compares against	the NEXT clock edge	the SAME clock edge
uses the period	yes	no - not at all
signed off at	the SLOW corner	the FAST corner
a violation means	the chip must run slower	the chip does not work

A trap worth naming

If you propagate only the max delay, your hold check silently uses the wrong number and reports violations that are not there - or misses ones that are.

The engine in the lab keeps both, and the hold report traces the MIN path.

What this looks like in a tool

Vivado: `report_timing -delay_type max` vs `-delay_type min`.

PrimeTime / OpenSTA: `-setup` vs `-hold`, or `report_timing -delay_type min_max` for both at once.

Every real STA tool does exactly this. The two-number rule is not a simplification - it is the whole reason setup and hold can disagree.



It Is Always the Same Six Things

Reading a timing report - it is always the same six things

Startpoint: a_q[3]_reg (rising edge-triggered by clk)
Endpoint: acc[7]_reg (rising edge-triggered by clk)
Path Group: clk Path Type: max (setup)

incr	arrival	point
0.000	0.000	clock clk (rise edge)
0.145	0.145	a_q[3]_reg/Q (DFF)
0.117	0.262	u21/Y (XOR2)
0.088	0.350	u34/Y (AND2)
0.041	0.391	acc[7]_reg/D (DFF)
		data arrival time 0.391
		clock period 1.050
		clock uncertainty -0.050
		library setup -0.090
		data required time 0.910
		SLACK (MET) +0.519

WHERE it starts

the launch flop - if it is a port, your input delay is in play

WHERE it ends

the capture flop, and the clock that captures it

HOW it got there

one line per cell, with the incremental delay

HOW LONG it had

period, minus uncertainty, minus setup

THE ANSWER

slack. Everything above exists to produce this number.

Read it bottom-up: the slack tells you IF, the incr column tells you WHERE.

Read it bottom-up: the slack tells you IF there is a problem, the incr column tells you WHERE it is.



The Four Numbers To Look At First

Look at	If it says	Then
SLACK	a positive number	this path is fine - look at the next one
SLACK	a negative number	the size tells you how hard the fix is
Startpoint	a port, not a flop	your input delay is in play - check it first
the largest incr	one cell dominating	that cell or its load is the problem
the largest incr	fifty small cells	the path is too deep - restructure or pipeline
TNS	roughly equal to WNS	one bad path - a local fix
TNS	hundreds of times WNS	systemic - the target or the architecture is wrong
unconstrained	anything but zero	fix that before believing any other number

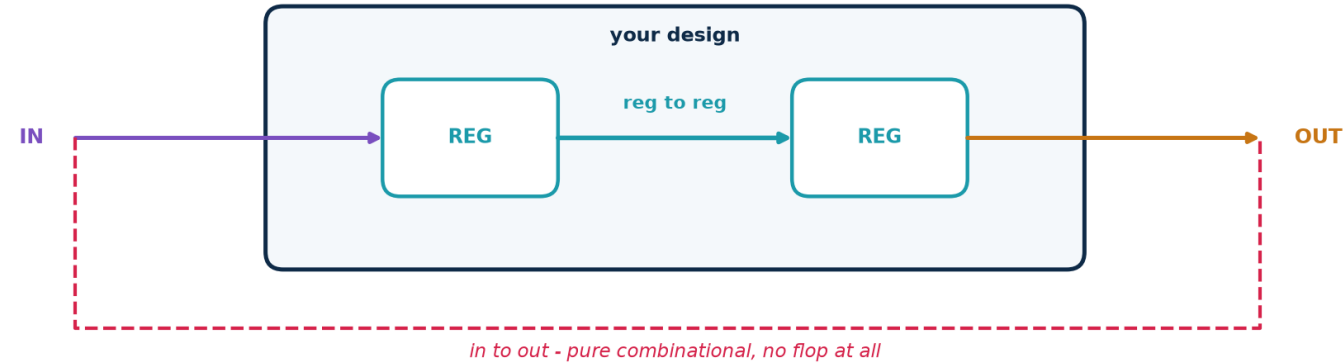
A habit that will make you fast

Do not read the whole report. Read the slack, then the startpoint, then the biggest incr line. Three numbers will tell you what kind of problem you have in about twenty seconds.



Four Kinds of Path, and Every Design Has All Four

Four kinds of path - and every design has all four



path group	what it is	constrained by	in practice
reg -> reg	the internal logic	create_clock	most of your paths
in -> reg	the world into you	set_input_delay	forgotten most often
reg -> out	you into the world	set_output_delay	forgotten second most
in -> out	straight through	both, on the same path	rare, and usually a mistake

A report that only shows reg-to-reg paths is a report on a quarter of your design.

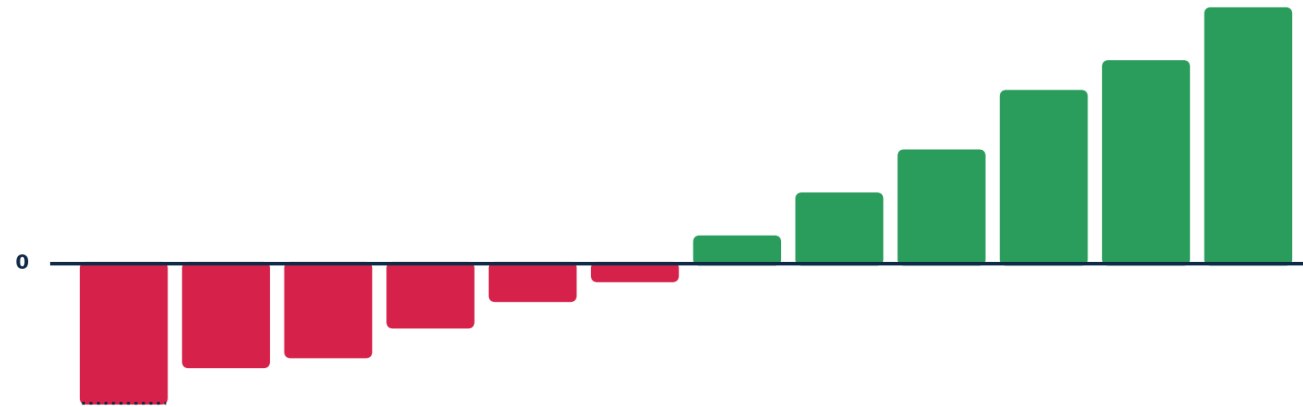
A report that only shows reg-to-reg paths is a report on a quarter of your design.



Two Numbers That Say Different Things

WNS and TNS say different things - you need both

slack of every endpoint, sorted



WNS = -0.42 ns
the single worst path

WNS -0.42, TNS -0.42

one bad path. Fix that path
and you are done today.

TNS = -1.36 ns over 6 endpoints
the size of the whole problem

WNS -0.42, TNS -180

hundreds of bad paths. The
target itself is wrong.

Use them together or you will misjudge the work

WNS tells you how bad the worst path is. TNS tells you how many paths are bad. WNS -0.42 with TNS -0.42 is an afternoon; WNS -0.42 with TNS -180 is an architecture review.



One Path Sets the Speed of the Entire Chip

Fmax - the number the datasheet quotes

$$F_{\max} = 1 / (\text{longest path delay})$$

longest path = clock-to-Q + logic + setup + uncertainty - skew

longest path	Fmax	design
2.196 ns	455 MHz	16-bit ripple-carry adder
4.094 ns	244 MHz	32-bit ripple-carry adder
2.315 ns	432 MHz	the same 32-bit adder, pipelined in two
1.939 ns	516 MHz	32-bit a + b, delay-oriented mapping

One path sets the speed of the entire chip

Not the average path. Not the second worst. The single longest one. Everything else in the design could be twice as fast and the number would not move.

This is why timing work is always about ONE path at a time - the critical path.

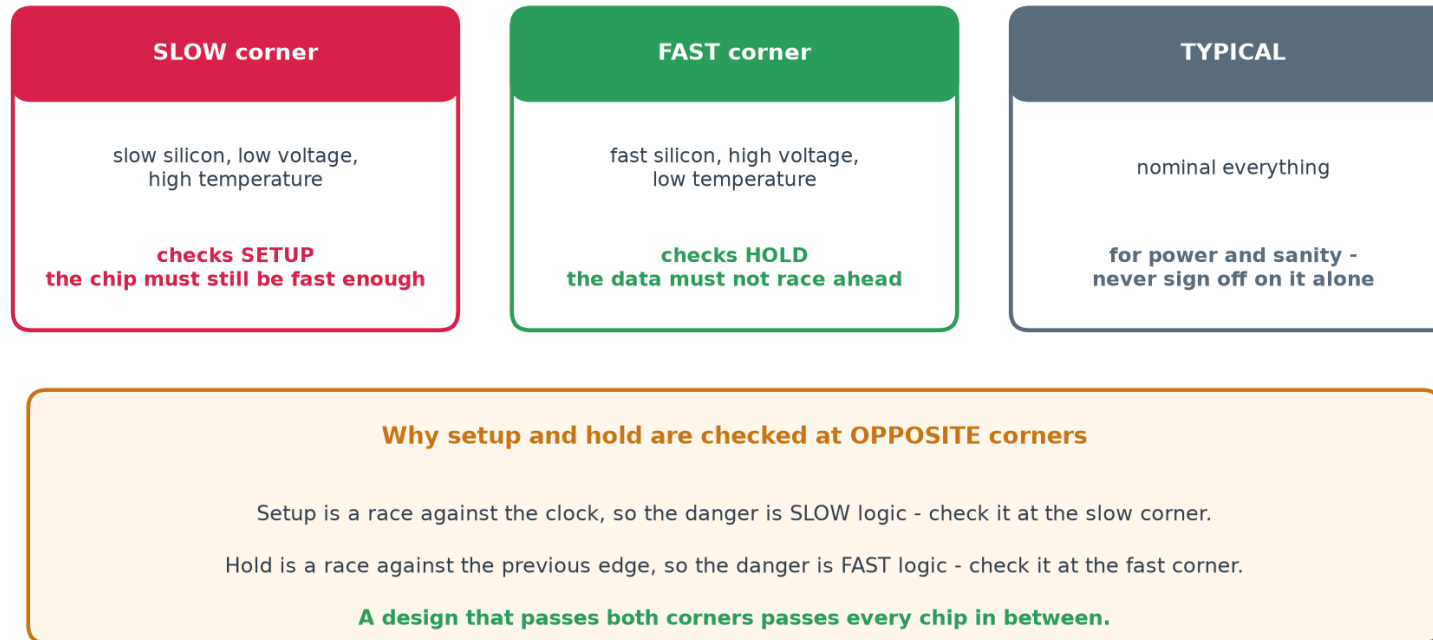
Not the average path. Not the second worst. The single longest one - which is why all timing work is about one path at a time.



One Netlist, Several Silicons

One netlist, several silicons - PVT corners

The same chip is slower when it is hot and starved of voltage, and faster when it is cold.



Setup is a race against the clock, so check it where the logic is SLOW. Hold is a race against the previous edge, so check it where the logic is FAST.



The Menu, Cheapest First

A setup violation - the fixes, cheapest first

Work down this list. Never start at the bottom.

- 1 Check the constraint first**
Is the period real? Is a false path missing? Half of all "violations" are the constraint file being wrong.
 - 2 Let the tool try harder**
Higher effort, delay-oriented mapping. In the lab this alone took a 32-bit adder from 4.6 ns to 1.9 ns.
 - 3 Restructure the logic**
Balance a deep chain into a tree. Same function, $\log(N)$ depth instead of N .
 - 4 Pipeline it**
Cut the path with a register. Throughput stays, latency grows by a cycle. The biggest single win available.
 - 5 Retime**
Move an existing register to a better place in the same logic. Latency does not change at all.
 - 6 Change the architecture**
A different adder, a different encoding, fewer operations per cycle. Real work, real payoff.
 - 7 Slow the clock down**
Always available, always last. You are shipping a slower chip.
- Steps 1 and 2 cost you minutes. Step 7 costs your customer performance for ever.**

Steps 1 and 2 cost you minutes. Step 7 costs your customer performance for ever.



Check the Constraint Before You Touch the Design

The report says	Suspect first	How to confirm in one minute
a huge violation on a reset path	a missing false path	look at the startpoint - is it rst_n?
every I/O path fails by the same amount	input/output delay too large	compare it against the upstream datasheet
a path you know is slow fails	a missing multicycle path	does the capture register have an enable?
a path across two clocks fails	missing set_clock_groups	are the two clocks actually related?
WNS is exactly -(period)	an undeclared generated clock	report_clocks - is anything missing?
nothing fails but you do not believe it	unconstrained endpoints	report the count - it should be zero

Roughly half of all reported violations are not design problems

They are constraint problems. Fixing the design first is the single biggest waste of time in timing closure - you can spend a week pipelining a path that was never real.



Let the Tool Try Harder - the Measured Result

one flag, 2.4x the frequency

```
# the SAME RTL:  assign {cout, sum} = a_q + b_q;

$ ABC=abc      make fmax      # default: area-oriented mapping
  longest path 4.615 ns  ->    217 MHz

$ ABC='abc -fast' make fmax    # delay-oriented mapping
  longest path 1.939 ns  ->    516 MHz

# and the hand-written ripple-carry chain, for comparison:
  longest path 4.094 ns  ->    244 MHz    (area mapping)
```

The result nobody expects

Under area-oriented mapping, the plain `a + b` is SLOWER than the hand-written ripple chain: 4.615 ns against 4.094 ns.

Both netlists were proved equivalent to the RTL over 400 random vectors before these numbers were quoted.

What it means

"Describe intent, not structure" is only half the rule.

The other half: check what your tool did with it. The same source can be a slow design or a fast one depending on one option you did not set.

You will reproduce both numbers yourself in lab T5.



6b · FIX 3

Restructuring - a Chain Is Depth N, a Tree Is Depth $\log N$

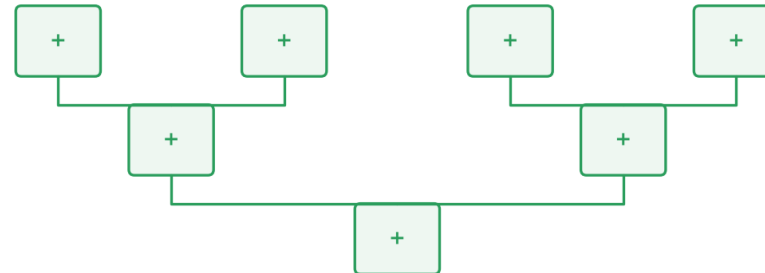
Restructuring - a chain is depth N, a tree is depth $\log N$

CHAIN: $a + b + c + d + e + f + g + h$ as written



depth 7

TREE: $((a+b)+(c+d)) + ((e+f)+(g+h))$ same answer



depth 3

Synthesis usually does this for you - usually

Associativity lets the tool rebalance a sum. It will NOT rebalance across a signal you assigned to an intermediate wire that something else also reads - that reader pins the structure.

What stops the tool doing it for you

Assign an intermediate sum to a named wire that something else also reads, and that reader pins the structure: the tool can no longer rebalance across it. If a sum must be fast, do not publish its intermediate results.



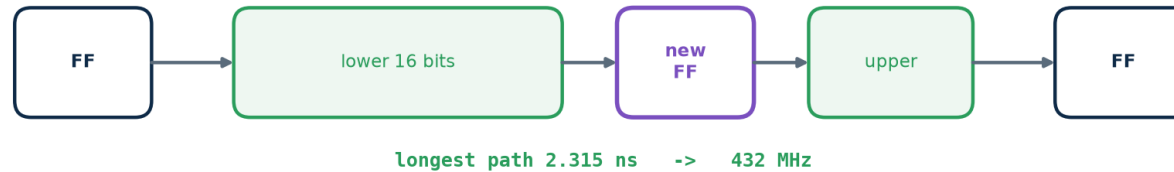
Pipelining - Cut the Path, Keep the Work

Pipelining - cut the path, keep the work

BEFORE - one long combinational path



AFTER - the same logic, cut in half by a register



what you gain

1.8x the clock frequency, and
the same result every cycle once
the pipe is full

what you pay

one extra cycle of latency, one
more register bank, and every
control signal must be delayed too

The biggest single win available - and the one with a real cost: one more cycle of latency, and every control signal alongside the data must be delayed to match.



Pipelining In RTL - and the Mistake To Avoid

rtl/add_ripple_pipe.v

```
// BEFORE - one stage, the whole 32-bit carry chain in one cycle
always @(posedge clk) {cout, sum} <= a_q + b_q;

// AFTER - cut at bit 16. Note that a_q[31:16] and b_q[31:16] must be DELAYED
// to meet the carry when it arrives, or the upper half adds the wrong operands.
always @(posedge clk) begin
    {cmid_q, sl_q} <= a_q[15:0] + b_q[15:0];    // stage 1
    ah_q <= a_q[31:16];                        // <- the easy line to forget
    bh_q <= b_q[31:16];

    {cout, sum} <= {ah_q + bh_q + cmid_q, sl_q};    // stage 2
end
```

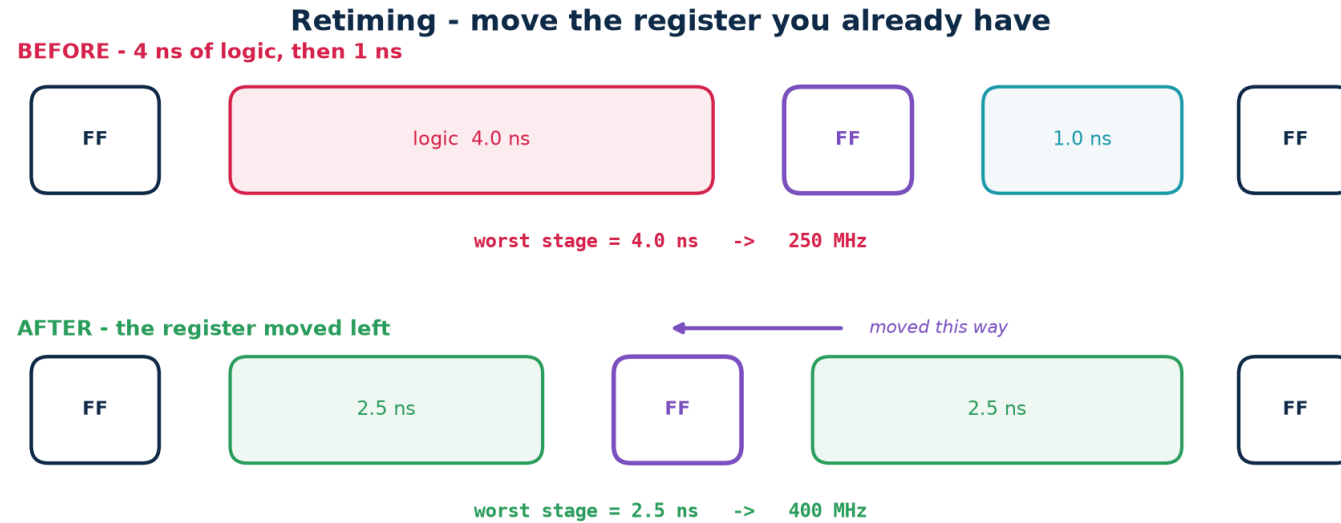
The classic pipelining bug, in one sentence

You cut the datapath and forget to delay something that travels beside it - an operand, a valid flag, a write enable. The design still meets timing beautifully and computes the wrong answer. Lab T5 makes you find this one deliberately.



6b · FIX 5

Retiming - Move the Register You Already Have



Retiming is free in a way pipelining is not

No register was added, so the latency is unchanged and no control signal needs re-aligning. The tool can do this for you (Vivado: `-retiming`; Yosys has no equivalent yet) - but only if your RTL lets it: a register with an asynchronous reset or a fanout to a port usually cannot move.

Free in a way pipelining is not

No register is added, so latency does not change and no control signal needs re-aligning. Vivado does it with `synth_design -retiming`; most ASIC tools have an equivalent. A register with an asynchronous reset, or one whose output goes straight to a port, usually cannot move.



Eight Questions Before We Move On

#	Question	The answer in one line
1	What does STA propagate forward?	arrival time - the latest, for setup
2	And backward?	required time, from period minus setup
3	Why does it need no stimulus?	it measures path length, not path activity
4	What is the one weakness of that?	it reports paths that can never switch - false paths
5	WNS -0.1, TNS -95. Diagnosis?	systemic; many paths just fail - review the target
6	Which corner signs off setup?	slow: slow silicon, low V, high T
7	Cheapest fix for a setup violation?	check the constraint, then the mapping effort
8	What does pipelining cost?	a cycle of latency, and every parallel signal must match

Part 6c is where the two failures separate: setup and hold need opposite fixes, and confusing them wastes days.

PART 6c

Setup and Hold Violations, and Their Resolution

Two failures that look similar in a report and could not be more different in the silicon.

- Setup: the data was too late. The chip runs slower.
- Hold: the data changed too soon. The chip does not work at any speed.
- Clock skew, jitter, and why hold problems appear only after layout
- A decision procedure you can follow under deadline pressure



Setup and Hold Are Opposite Problems

Setup and hold are opposite problems

SETUP

the data was too LATE

Checked against	the NEXT clock edge
Uses the period	yes
Caused by	too much logic between flops
Fixed by	less logic, or a slower clock
Slowing the clock	helps
Found at	synthesis, then again after layout

HOLD

the data changed too SOON

Checked against	the SAME clock edge
Uses the period	no - not at all
Caused by	too little logic, plus skew
Fixed by	ADDING delay to the data path
Slowing the clock	does nothing whatever
Found at	layout, when skew is finally known

A chip with a setup violation runs slower.

A chip with a hold violation does not work at any speed.

A chip with a setup violation runs slower. A chip with a hold violation does not work at any speed. That difference decides everything you do next.



Side By Side, With the Difference Circled

everything in part 6c follows from these two

```
SETUP    required = period + skew - setup - uncertainty
         arrival  = clk_to_q + logic
         slack    = required - arrival

HOLD     required = skew + hold + uncertainty
         arrival  = clk_to_q + logic      <- the SHORTEST path, not the longest
         slack    = arrival - required

# the period appears in the setup equation and NOWHERE in the hold equation
# the subtraction is the other way round: setup wants arrival SMALL,
# hold wants it LARGE
```

Consequence 1

Slowing the clock fixes setup and does nothing at all for hold.
Speeding it up breaks setup and, again, does nothing to hold.

Consequence 2

Adding logic fixes hold and breaks setup.

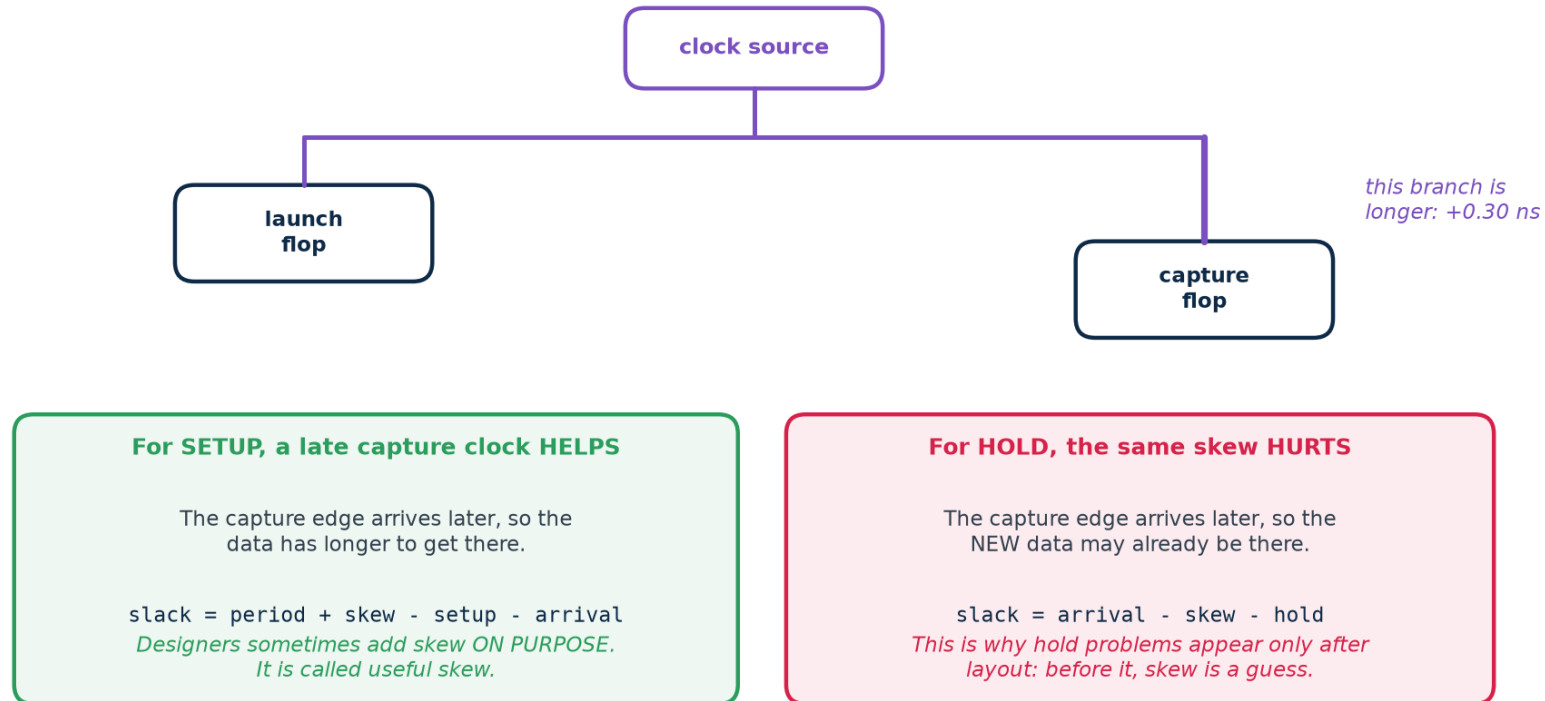
Removing logic fixes setup and can break hold. Every fix trades one against the other.



The Clock Does Not Arrive Everywhere At Once

Clock skew helps setup and hurts hold

The clock does not arrive everywhere at once. The difference between two registers is SKEW.



Skew that makes the capture edge LATE helps setup and hurts hold. Designers sometimes add it deliberately - it is called useful skew.



Where the Numbers Come From, and When You Know Them

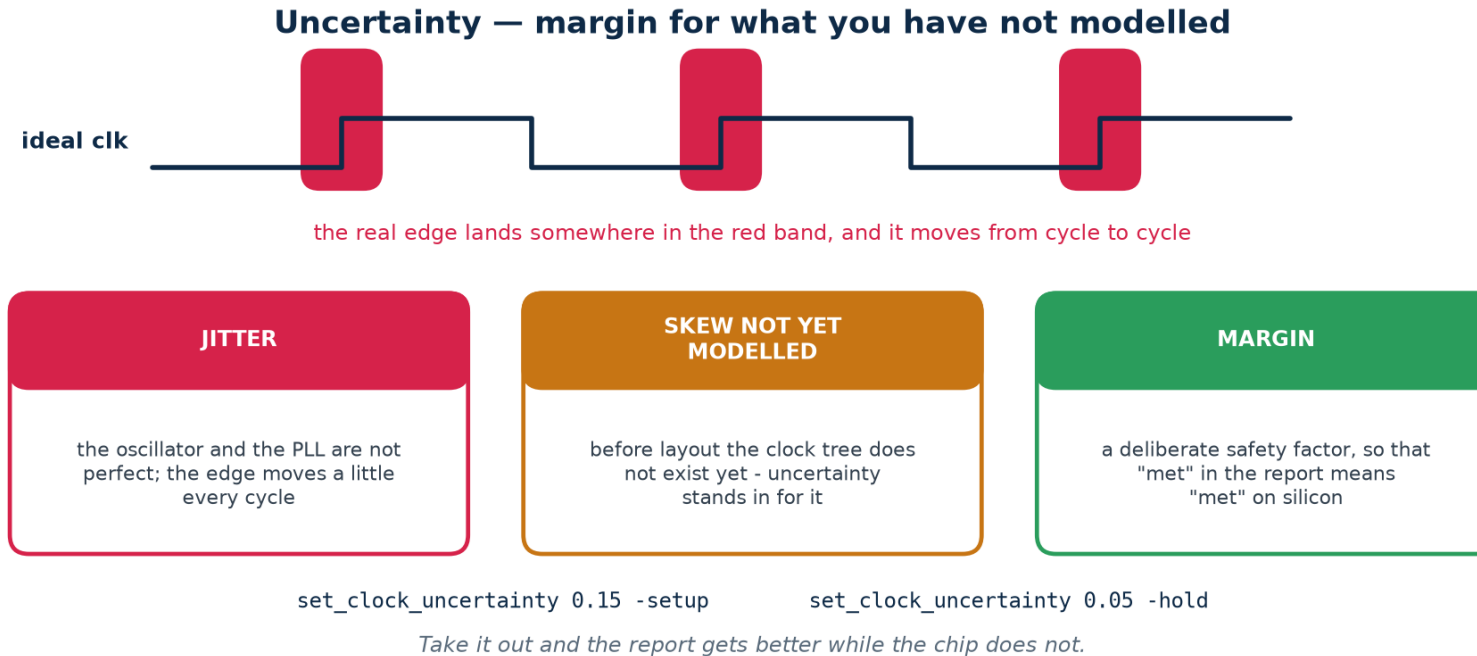
Effect	What causes it	Known when?	Modelled by
Clock skew	different clock-tree branch lengths	after clock-tree synthesis	the real tree, or uncertainty before it
Clock jitter	PLL and oscillator imperfection	from the datasheet	set_clock_uncertainty, both checks
Duty-cycle distortion	unbalanced buffers in the tree	after layout	matters for both clock edges
On-chip variation	process gradients across the die	statistically	derating factors, or extra uncertainty
Crosstalk	a neighbouring net switching	after routing	signal-integrity analysis, post-route

Why hold violations always seem to arrive late in a project

Before layout there is no clock tree, so there is no real skew - and the hold check has almost nothing to fail against. The moment the tree is built, the skew becomes real and hold violations appear all at once. This is normal, and it is why hold is a place-and-route problem far more often than an RTL one.



The Margin That Stands In For What You Cannot See Yet



Stage	Typical setup uncertainty	Why
post-synthesis (FPGA)	0.10 - 0.20 ns	no clock tree exists; jitter unknown
post-place-and-route	jitter only, 0.02 - 0.05 ns	the tool now uses the real skew
ASIC, pre-CTS	0.15 - 0.30 ns	plus a margin for OCV
ASIC, sign-off	jitter + OCV derating	the tree is measured

Take the uncertainty out and the report improves while the chip does not.



What They Look Like, and What Causes Them

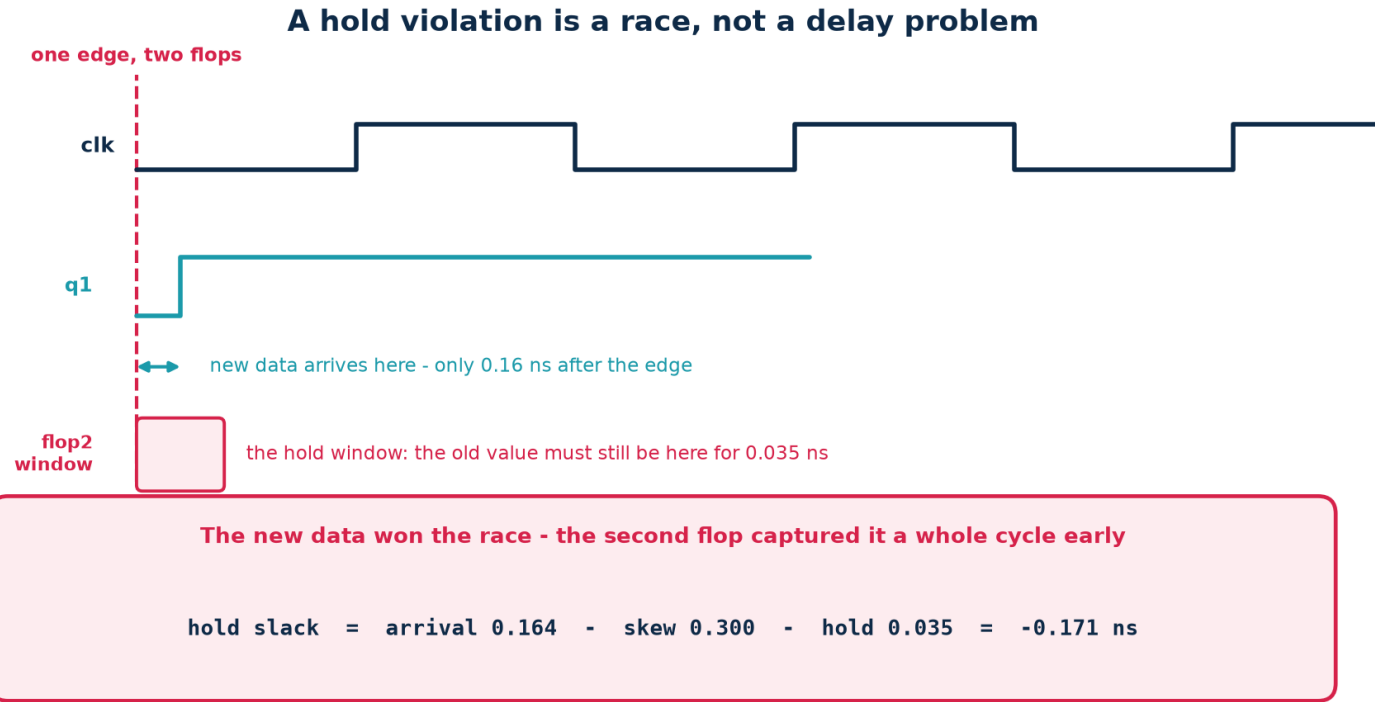
Symptom in the report	Most likely cause	First thing to try
one path fails, the rest are fine	a genuinely long path	look at its incr column
one cell has a huge incr	high fanout, or a weak driver	set_max_fanout, or let the tool buffer it
fifty small cells in the path	the logic is too deep	restructure, then pipeline
every path fails by a similar amount	the period is too aggressive	sanity-check the target against the technology
fails only after place-and-route	routing delay, not logic	floorplanning; keep the path local
fails only at the slow corner	correct behaviour	that is the corner you sign off at

The number that tells you how much work it is

A slack of -0.05 ns on a 5 ns period is a 1% problem - the tool will often find it with a higher effort setting. A slack of -2.5 ns on the same period is a 50% problem: no option will close that, and you are looking at pipelining or a different architecture.



A Race, Not a Delay Problem



Notice what is NOT in that equation: the clock period.

You can run this chip at 1 Hz and it will still be broken. A hold violation is a functional failure, not a performance one - the only fix is to make the DATA path slower.

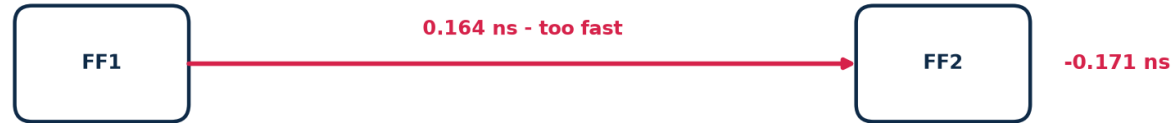
Notice what is not in the hold equation: the clock period. You can run this chip at 1 Hz and it is still broken.



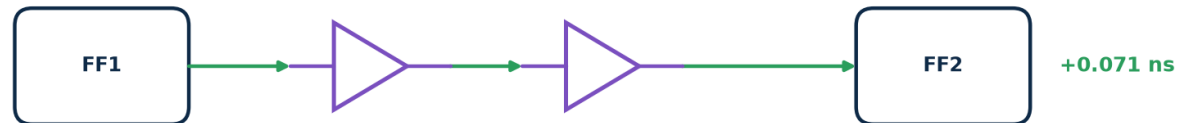
Add Delay, On Purpose

Fixing hold - add delay, on purpose

BEFORE - nothing between the two flops



AFTER - two buffers inserted in the data path



delay cells - their only job is to waste time

You almost never do this by hand

Place-and-route inserts hold buffers automatically, after layout, when the real skew is known. Your job in RTL is to not make it impossible: keep clock domains clean, and never gate a clock by hand when an enable would do.

Adding delay costs area and power but no frequency - which is why hold is always fixable.

Measured in the lab

hold_demo with 0.30 ns of skew: -0.165 ns, VIOLATED. The same design with two delay cells in the data path: +0.071 ns, MET. Nothing else changed - not the clock, not the function.



Who Actually Fixes It - and What Your RTL Must Not Do

Place-and-route does the fixing

After layout, when the real skew is known, the tool inserts hold buffers automatically.

It has the one thing you do not: the actual clock-tree delays.

Hold fixing costs area and power, never frequency - which is why it is always possible.

What RTL can do to make it impossible

Gate a clock by hand instead of using an enable - it adds skew you cannot control.

Cross clock domains without a synchroniser - no amount of buffering fixes that.

Use both clock edges in the same path - it halves the hold budget.

The one hold fix that belongs in RTL

A proper two-flop synchroniser on every asynchronous crossing. That is not a hold fix in the STA sense - it is what stops the crossing being a timing question at all.



The Numbers You Will Produce Yourself

The numbers you will measure yourself in the lab

Every figure below came out of the STA engine you are about to build.

design / experiment	worst slack	result	what it teaches
32-bit ripple, area mapping	4.094 ns	244 MHz	the baseline
32-bit a+b, area mapping	4.615 ns	217 MHz	SLOWER - see below
32-bit a+b, delay mapping	1.939 ns	516 MHz	same RTL, 2.4x faster
hold demo, 0.30 ns skew	-0.165 ns	broken	fails at any frequency
hold demo, delay added	+0.071 ns	works	two cells fixed it
slow path, no exception	-1.193 ns	false alarm	the tool was misled
slow path, multicycle 4	+0.392 ns	real	one SDC line

Read rows 2 and 3 again - this is the lesson of the whole topic

The SAME RTL, a + b, ran at 217 MHz and at 516 MHz. Nothing in the source changed.
Only the synthesis option did.

"Describe intent, not structure" is only half the rule.
The other half is: check what your tool did with it.

Nothing quoted in this topic is a claim you cannot reproduce with make.



How the Hold Demo Adds Its Delay, and Why

what the lab actually does, and what it is standing in for

```
// rtl/hold_fixed.v
// dly_sel is a real input port, tied to 0 by the testbench.
assign d0 = q1 ^ dly_sel[0];    // two XOR gates in the data path
assign d1 = d0 ^ dly_sel[1];    // functionally a no-op when dly_sel == 0

// Why not a chain of buffers with (* keep *) ?
//   opt_clean deletes them. keep_hierarchy plus -flatten deletes them too.
//   Only logic that a real port can influence survives optimisation.
```

What this demonstrates honestly

That adding delay to the data path turns a hold violation into a met one, and by how much.

That the tool cannot be asked politely to keep useless logic.

What it is NOT

A style to copy. In a real flow, place-and-route inserts hold buffers with knowledge of the actual clock tree.

Writing XOR gates into your datapath to fix hold is not engineering - it is a teaching device, and it is labelled as one.

Being told what a demonstration cannot show is part of the demonstration.



What To Do When the Report Is Red

Step	Do this	Because
1	Read the unconstrained-endpoint count first	a clean report on half a design is worthless
2	Is it setup or hold? Look at the path type	the two need opposite fixes
3	Read the startpoint and endpoint by name	it often names the problem for you
4	Ask whether the path is real	half of all violations are constraint bugs
5	Look at the largest incr in the path	one big cell or fifty small ones - different fixes
6	Compare WNS with TNS	one path, or the whole design?
7	Apply the cheapest fix that could work	and re-run before trying the next one
8	Change ONE thing per run	or you will not know what helped

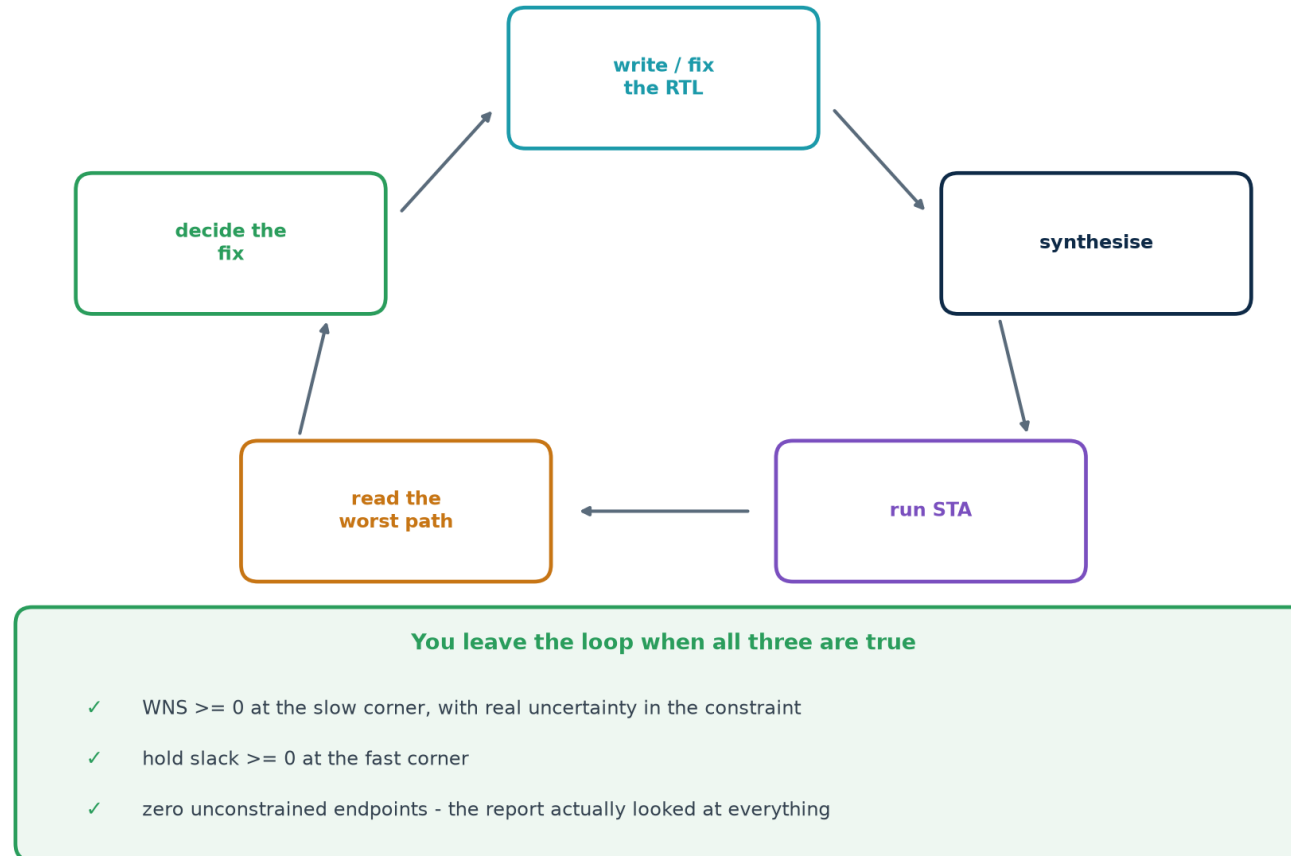
Step 8 is the one people skip

Under deadline pressure it is tempting to change the period, the effort level and the RTL all at once. When the slack moves you will have no idea which change did it - and no idea what to undo when it moves the wrong way.



Timing Closure Is a Loop, and You Exit It Once

Timing closure is a loop, and you exit it once



A design is closed when WNS ≥ 0 at the slow corner, hold ≥ 0 at the fast corner, and the unconstrained count is zero. Two out of three is not closed.



Ten Ways To Get a Green Report and a Broken Chip

#	The mistake	What it costs you
1	No constraint file at all	every path unchecked, report meaningless
2	Clock declared, I/O not	every boundary path unchecked
3	A generated clock never declared	a whole domain unchecked
4	set_false_path used to silence a real path	a field failure
5	Multicycle setup without the hold line	a hold violation you created
6	Multicycle claimed without an enable in the RTL	wrong data captured
7	Uncertainty reduced to make the report pass	no margin on silicon
8	Signing off setup at the typical corner	fails on slow silicon
9	Ignoring hold because the clock is slow	fails at every speed
10	Never checking the unconstrained count	all of the above, silently

Nine of these ten produce a report that says everything is fine.



Eight Questions Before the Labs

#	Question	The answer in one line
1	Setup violation - what does the chip do?	works, but only at a lower frequency
2	Hold violation - what does it do?	does not work at any frequency
3	Does the period appear in the hold equation?	no - that is the whole point
4	Late capture clock: helps which check?	setup; it hurts hold by the same amount
5	Why do hold problems appear after layout?	before it, there is no clock tree, so no real skew
6	How is hold fixed?	by ADDING delay to the data path
7	What does that cost?	area and power, never frequency
8	Setup at which corner, hold at which?	setup at slow, hold at fast

Answer these eight and the labs will feel like confirmation rather than discovery.

TOOLS

Installing and Running the Timing Flow

Everything in this topic runs on free software. The vendor tools the syllabus names are covered too - as the last step, not the first.

- The landscape: Vivado, Quartus, PrimeTime, Tempus, OpenSTA, and yours
- Tier 1 install: one apt line, and every exercise runs
- The lab flow: RTL in, slack out - with no black box in between
- The same four steps in Vivado, and how SDC maps onto XDC



What Exists, and What You Will Use

Timing tools - what exists, and what you will use

tool	used for	licence	the timing command
Vivado (Xilinx/AMD)	FPGA	free WebPACK edition	report_timing_summary
Quartus Prime (Intel)	FPGA	free Lite edition	TimeQuest / Timing Analyzer
PrimeTime (Synopsys)	ASIC sign-off	commercial	the industry reference
Tempus (Cadence)	ASIC sign-off	commercial	the other reference
OpenSTA	ASIC / academic	open source	reads real .lib and .sdc
OpenROAD / OpenLane	full ASIC flow	open source	OpenSTA inside it
sta.py (this course)	teaching	you write it	200 lines, no black box

They all do the same arithmetic

Arrival, required, slack. The differences are the delay models, the report formats and the price.
Learn to read one report and you can read all of them.

The syllabus names Vivado and ModelSim, so both are covered here - but every exercise also runs with no licence at all, on the open-source flow.

They all compute arrival, required and slack. The differences are the delay models, the report format and the price.



Tier 1 - One Line, and Every Exercise Runs

Install this, and every exercise in the topic runs

One apt line. No licence, no account, no download manager.

TIER 1 - required

Debian / Ubuntu / WSL2

```
sudo apt update
```

```
sudo apt install yosys iverilog gtkwave python3 python3-matplotlib
```

Yosys synthesis, and the JSON netlist that sta.py reads

Icarus simulation, to prove the optimised netlist still works

GTKWave waveform viewer, for the hold-violation exercise

Python 3 runs sta.py - the timing engine you build in part C

matplotlib draws the Fmax curves in the closure exercise

```
yosys -V && iverilog -V && python3 -c "import matplotlib"
```

Three version strings and you are ready. On Windows, install WSL2 first and run the same line inside it.

Debian / Ubuntu / WSL2 — copy this line

```
sudo apt update
sudo apt install yosys iverilog gtkwave python3 python3-matplotlib
yosys -V && iverilog -V && python3 -c "import matplotlib; print('ok')"
```

On Windows: install WSL2 first (wsl --install), then run the same three lines inside it.



Tiers 2 and 3 - For the Last Four Exercises Only

Two optional tiers - for the last exercises only

TIER 2 - recommended

a real STA tool that reads real Liberty and SDC

```
git clone https://github.com/parallaxsw/OpenSTA  
  
cd OpenSTA && mkdir build && cd build && cmake .. && make -j4  
  
sudo make install  
  
OpenSTA    the same command set as PrimeTime, no licence at all
```

TIER 3 - vendor

what the syllabus names; large, slow, licence-gated

Vivado ML Edition (WebPACK) free, about 40 GB, Linux or Windows

ModelSim / Questa Intel Starter free, simulation only

Register for a free account, then run the web installer and select only your device family.

Exercises G1-G5 use these. Everything before them - 57 of the 62 exercises - runs on tier 1 alone.

Nothing in this course is gated behind a licence you cannot get.

Nothing in this course is gated behind a licence you cannot get

57 of the 62 exercises run on tier 1 alone. Tier 2 (OpenSTA) lets you check your engine against a real one. Tier 3 (Vivado, ModelSim) is there because the syllabus names them and because you will meet them at work.



Installing Vivado, Step By Step

Step	What to do	Watch out for
1	Create a free AMD/Xilinx account	the download will not start without it
2	Download the Unified Installer (web installer, ~200 MB)	not the full 40 GB archive
3	Choose "Vivado ML Edition" then "Vivado ML Standard"	Standard is the free one - no licence file needed
4	Deselect every device family except Artix-7	this cuts 40 GB to about 8 GB
5	On Linux, install the cable drivers afterwards	only needed if you program a real board
6	Verify: vivado -version	then everything below runs headless

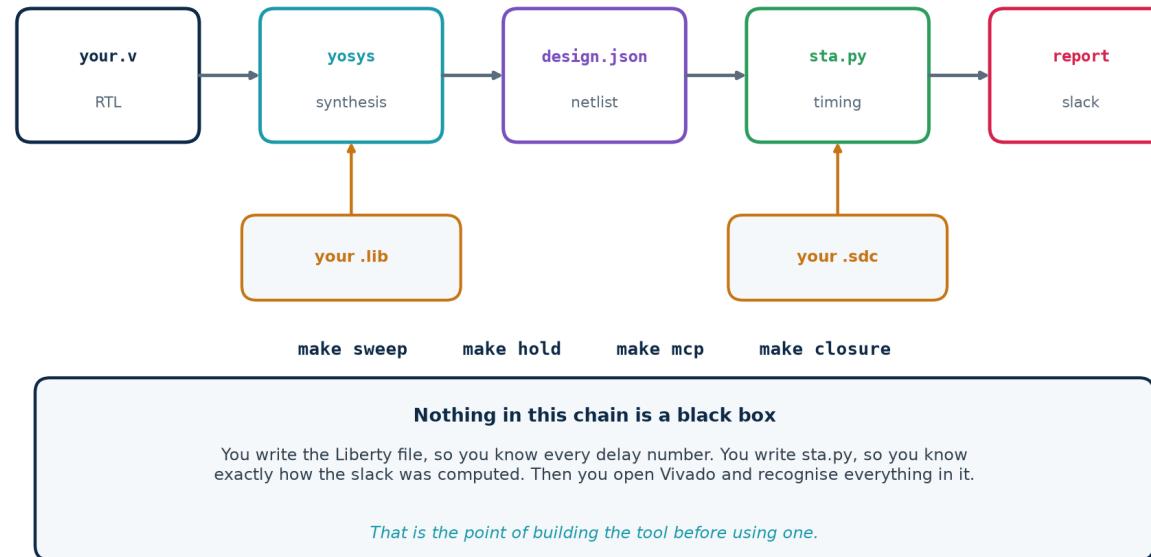
You do not need a board

Every timing exercise in this topic is a synthesis-and-report exercise. Vivado will happily target a device it has never seen and give you a real timing report for it.



RTL In, Slack Out - With No Black Box In Between

The lab flow - RTL in, slack out



```
make lib      # read the Liberty file and print the cell table
make tiny     # three flops - a report you can check by hand
make sweep    # Fmax against width, for three adder styles
make hold     # a hold violation, then the same design fixed
make mcp      # a false violation, then the multicycle path that removes it
make verify   # prove every optimised netlist still computes the right answer
```



The Same Four Steps, In Vivado

The same four steps in Vivado

1

read_verilog rtl/add32.v

load the design

2

read_xdc constraints/vivado.xdc

load the constraints - the same create_clock you wrote

3

synth_design -top add32 -part xc7a35tcpg236-1

synthesise for a real device

4

report_timing_summary -file post_synth_timing.rpt

the report - WNS, TNS, and the worst path

5

opt_design ; place_design ; route_design

and again after layout, when skew is real

Run it headless: vivado -mode batch -source vivado_timing.tcl

The script is in the lab. Every line above is the industrial equivalent of one line in your Makefile - the concepts do not change, only the command names.

Every line is the industrial equivalent of one line in your Makefile. The concepts do not change - only the command names.



The Script, In Full

```
# scripts/vivado_timing.tcl - run with: vivado -mode batch -source this.tcl
set part xc7a35tcpg236-1

read_verilog [glob rtl/*.v]
read_xdc      constraints/vivado.xdc
synth_design -top add32 -part $part -flatten_hierarchy rebuilt

report_timing_summary -file rpt/post_synth_summary.rpt
report_timing -delay_type max -max_paths 10 -file rpt/post_synth_setup.rpt
report_timing -delay_type min -max_paths 10 -file rpt/post_synth_hold.rpt

opt_design ; place_design ; route_design
report_timing_summary -file rpt/post_route_summary.rpt

# the number to compare against your own engine:
puts "WNS = [get_property SLACK [get_timing_paths -delay_type max]]"
```

The comparison that makes the whole topic land

Run your engine and Vivado on the same RTL. The absolute numbers will differ - different technology, different delay model. The STRUCTURE of the report, the critical path it names, and the effect of every constraint change will be the same.



The Same Language, Two Names

SDC and XDC - the same language, two names

command	SDC	XDC	note
create_clock	yes	yes	identical
set_input_delay	yes	yes	identical
set_output_delay	yes	yes	identical
set_false_path	yes	yes	identical
set_multicycle_path	yes	yes	identical
set_clock_uncertainty	yes	yes	Vivado also derives its own
set_property PACKAGE_PIN	no	yes	XDC extension - pin placement
set_load / set_driving_cell	yes	partly	ASIC-oriented

XDC = SDC + Xilinx physical constraints in one file. The timing half is standard SDC.

Write the timing constraints once and they travel between tools almost unchanged.

```
# constraints/vivado.xdc - the timing half is identical to the SDC file
create_clock -name clk -period 5.000 [get_ports clk]
set_input_delay -clock clk -max 1.20 [get_ports {a[*] b[*]}]
set_property PACKAGE_PIN W5 [get_ports clk] # <- the XDC-only half
set_property IOSTANDARD LVCMOS33 [get_ports clk]
```

LABS T1-T7

The Practical Component

25 hours of hands-on work: build the analyser, then use it on real violations, then prove the ideas transfer to an industrial tool.

- Syllabus: Timing Analysis and Closure Labs (10 h)
- Syllabus: Design Synthesis and Optimisation Labs (15 h)
- 62 graded exercises, every one with a worked solution
- Nothing quoted in this topic that you cannot reproduce with make



Seven Parts, 25 Hours

The practical component - 25 hours, seven parts

Syllabus practicals: Timing Analysis and Closure Labs (10 h) + Design Synthesis and Optimisation Labs (15 h)

A	Build the delay model	write cda_edu.lib, read it with liberty.py	2 h
B	Build the timing graph	netlist to nodes and arcs; check it by hand	3 h
C	Arrival, required, slack	finish sta.py; verify against arithmetic	4 h
D	Constraints	write SDC; watch the report change	4 h
E	Setup closure	Fmax sweep, mapping options, pipelining	5 h
F	Hold and exceptions	skew, hold fixes, multicycle paths	4 h
G	Vivado / OpenSTA	the same design in an industrial tool	3 h
62 graded exercises, every one with a worked solution Parts A-C build the tool. Parts D-F use it on real violations. Part G proves the ideas transfer. Nothing is quoted in this course that you cannot reproduce with make.			

Parts A-C build the tool. Parts D-F use it on real violations. Part G proves the ideas transfer.



Build the Delay Model, Then the Graph

T1 · The delay model (2 h)

Write lib/cda_edu.lib: 14 cells, each with an intrinsic delay, a load factor and an input capacitance.

Write sta/liberty.py to read it and print the cell table.

Deliverable: a table of 14 cells with every number accounted for.

Check: make lib

T2 · The timing graph (3 h)

Synthesise rtl/tiny.v with Yosys, write the netlist as JSON.

Turn the JSON into nodes and arcs: one node per pin, cell delay on the internal input-to-output arc.

Deliverable: the graph for a three-flop design, drawn on paper and printed by your code - and they agree.

Check: make tiny

The arc convention that makes reports readable

Charge the cell delay on the arc INSIDE the cell, using the load on its output net; charge clock-to-Q at the Q pin. Net arcs cost zero. Get this wrong and every report line is attributed to the wrong cell.



Arrival, Required, Slack - the Engine Itself

what your engine prints when T3 is finished

```
$ ./scripts/sta.sh tiny --paths 1 -p 1.0
```

```
Path 1  endpoint q_reg/D  (DFF)
        startpoint p_reg
        incr  arrival  pin
0.000    0.000    clock edge at p_reg
0.164    0.164    p_reg/Q          (DFF)
0.000    0.164    u97/B            (XOR2)
0.117    0.281    u97/Y            (XOR2)
0.000    0.281    q_reg/D          (DFF)
          0.910    required (period - setup)
          0.629    SLACK  MET

WNS (worst slack) : +0.629 ns  MET
longest path      : 0.371 ns    Fmax : 2693.2 MHz
```

The exercise that matters most in the whole topic

Compute +0.629 on paper before you run it. When your arithmetic and your code agree to three decimals, you understand static timing analysis - and no report from any vendor will ever be mysterious again.



Constraints - Watch the Report Change

Exercise	What you change	What you must observe
T4.1	the period, from 5 ns down to 1 ns	the slack falls linearly; the path does not change
T4.2	add set_input_delay for the first time	new paths appear in the report that were not there before
T4.3	remove it again	the unconstrained count goes up; the WNS improves. Explain that.
T4.4	add 0.15 ns of uncertainty	every setup slack drops by exactly 0.15
T4.5	add 0.30 ns of skew to one register	its setup slack improves and its hold slack collapses
T4.6	set_false_path on the critical path	the WNS jumps. Write down why that is dangerous.

T4.3 and T4.6 are the two that teach the most

Both make the report look BETTER while making the analysis worse. Learning to distrust an improving number is the professional skill this topic is really about.



Setup Closure - the Fmax Sweep

the experiment that shows what synthesis actually did

```
$ make sweep
```

```
=== hand-written carry chain ===      === a + b, delay-driven mapping ===
W  cells  longest(ns)  ns/bit      W  cells  longest(ns)  ns/bit
4   30     0.773      0.1933      4   33     0.831      0.2077
8   62     1.247      0.1559      8   74     1.183      0.1479
16  126     2.196      0.1373     16  159     1.547      0.0967
32  254     4.094      0.1279     32  332     1.939      0.0606
64  510     7.889      0.1233     64  681     2.318      0.0362
```

```
ripple:    ns/bit converges on 0.123  -> delay is LINEAR in width
a+b fast:  ns/bit keeps FALLING       -> the tool built a carry tree
```

What you must explain

Why ns/bit is constant for the ripple chain and falling for the tool's adder.
Why the plain a+b is SLOWER than the ripple chain under area mapping.

Then prove it still works

make verify runs all three netlists against a reference over 500 random vectors.

An optimisation you have not verified is not an optimisation.



Hold Violations and Exceptions

two violations, two different kinds of fix

```
$ make hold
WNS (worst slack) : -0.165 ns VIOLATED (hold_demo, 0.30 ns skew)
WNS (worst slack) : +0.071 ns MET (hold_fixed, same skew)

$ make mcp
WNS (worst slack) : -1.193 ns VIOLATED (slow_path, no multicycle)
WNS (worst slack) : +0.392 ns MET (slow_path, multicycle 4)
```

Exercise	Task
T6.1	Reproduce both violations, then explain in one sentence each why the fix works
T6.2	Increase the skew until hold_fixed also fails - how much delay would it need?
T6.3	Add the multicycle -setup line but NOT the -hold line. Predict, then measure.
T6.4	Remove the enable from slow_path.v. The SDC is now a lie - show what breaks.
T6.5	Write a false path that hides a real violation, then argue against yourself.



The Same Design In an Industrial Tool

Exercise	Tool	What you produce
T7.1	Vivado	the same add32, synthesised for an Artix-7, with your own XDC
T7.2	Vivado	post-synthesis and post-route timing summaries, compared
T7.3	Vivado	the multicycle path applied in XDC - same effect, vendor syntax
T7.4	OpenSTA	your netlist and a real .lib, read by a real STA tool
T7.5	both	a one-page comparison: what matched, what did not, and why

T7.5 is the deliverable that proves you learned the topic

The numbers will differ. The reasoning will not. If you can explain why Vivado's WNS differs from yours, and point at the delay model rather than shrugging, you can close timing on a real design.



How the 62 Exercises Are Weighted

Part	Exercises	Weight	Assessed on
A · delay model	6	8%	every number traceable to the library
B · timing graph	8	12%	hand-drawn graph matches the code
C · the engine	10	20%	hand arithmetic matches the report
D · constraints	12	15%	a defensible line-by-line SDC file
E · setup closure	12	20%	measured Fmax, plus verification
F · hold and exceptions	9	15%	correct diagnosis before the fix
G · industrial tools	5	10%	the comparison write-up

The rule that runs through every part

A fix you cannot explain is not a fix. Marks are given for the reasoning, not for the green report - because in this field it is trivially easy to produce a green report on a broken design.



Glossary - Terms Used In This Topic (1 of 2)

Term	Meaning
arrival time	when the data actually reaches a pin, measured from the clock edge
required time	when it had to be there by, for the check to pass
slack	required minus arrival. Positive fits, negative does not
WNS	worst negative slack - the single worst path in the design
TNS	total negative slack - every negative slack added up
setup time	how long D must be stable BEFORE the clock edge
hold time	how long D must stay stable AFTER the clock edge
clock-to-Q	how long after the edge before Q is valid
clock skew	the difference in clock arrival between two registers
clock jitter	cycle-to-cycle variation in the clock edge position
uncertainty	the margin that stands in for jitter and unmodelled skew
critical path	the path with the worst slack - the one that sets Fmax



Glossary - Terms Used In This Topic (2 of 2)

Term	Meaning
Fmax	1 divided by the longest path delay - the highest safe clock
endpoint	where a path ends: a flop's D pin, or an output port
startpoint	where it begins: a flop's Q pin, or an input port
path group	reg-to-reg, in-to-reg, reg-to-out, in-to-out
false path	a path the tool must not check, because it cannot be exercised
multicycle path	a path allowed more than one cycle to complete
SDC / XDC	the constraint language; XDC adds Xilinx physical constraints
Liberty (.lib)	the file that gives each cell its delay, setup and hold
PVT corner	a process/voltage/temperature combination to analyse at
retiming	moving an existing register across logic to balance stages
pipelining	adding a register to cut a long path in two
timing closure	the loop that ends when every check passes at every corner



Topic 6 In Ten Lines

- A flip-flop samples over a WINDOW around the clock edge, not at an instant.
- Setup means the data was too late. Hold means it changed too soon.
- slack = required - arrival, computed at every endpoint. That is all STA does.
- The clock period is in the setup equation and nowhere in the hold equation.
- An unconstrained path is not a passing path - it is an unexamined one.
- An exception is a promise. The tool believes you without checking.
- Half of all reported violations are constraint bugs, not design bugs.
- Setup is fixed by less logic or a slower clock; hold only by MORE delay.
- The same RTL ran at 217 MHz and 516 MHz. Only a synthesis option changed.
- Describe intent, not structure - then check what your tool did with it.



CLOSE

What To Do Next

Before the next session

Install tier 1 and run `make` in the lab directory. It should end with "Topic 6 lab complete".

Compute the +0.629 ns slack on paper. Bring your working.

Read one timing report - any tool, any design - and find its critical path.

Carry into every project after this

Write the constraint file before you write the RTL. It is the specification.

Check the unconstrained-endpoint count on every single run.

Never write an exception you cannot justify in one sentence.

Verify every optimisation. A faster design that computes the wrong answer is not faster.

The sentence to leave with

A timing report is not a verdict on your design. It is a verdict on your design AND your constraints - and only one of those two is usually the problem.